


```

1 ### Complete Novel_lecture_2 Pluto Notebook
2 ### Earth → L1 Minimum-Fuel Transfer + Interval Tube Width Growth
3 ### Paste into a new Pluto notebook or run cell by cell
4
5 # =====
6 # Cell 1: Imports
7 # =====
8 begin
9     using Pkg
10    Pkg.activate(".")
11    Pkg.instantiate()
12    using InfiniteOpt, Ipopt, Plots, NLSolve, LinearAlgebra
13    using IntervalArithmetic
14 end
15
16 # =====
17 # Cell 2: Parameters (Earth-Moon)
18 # =====

```

```

Activating project at `~/julia/pluto_notebooks`
Precompiling project...
  × OrdinaryDiffEqDefault
  × OrdinaryDiffEq
  × TaylorIntegration → TaylorIntegrationDiffEqExt
0 dependencies successfully precompiled in 20 seconds. 468 already precompiled.

The following 2 direct dependencies failed to precompile:

TaylorIntegrationDiffEqExt

Failed to precompile TaylorIntegrationDiffEqExt [21fdf329-9f0e-520d-ae26-dab38dab1a79] to "/Users/nitishtyagi20/.julia/compiled/v1.11/TaylorIntegrationDiffEqExt/jl_aRfteY".
ERROR: LoadError: UndefVarError: `supports_solve_rng` not defined in `SciMLBase`
Stacktrace:
 [1] getproperty(x::Module, f::Symbol)
   @ Base ./Base.jl:42
 [2] top-level scope
   @ ~/.julia/packages/OrdinaryDiffEqDefault/ym4p/src/default_alg.jl:50
 [3] include(mod::Module, _path::String)
   @ Base ./Base.jl:562
 [4] include(x::String)

```

```

1 begin
2   μ = 0.012150585609624
3   const u_max      = 0.1
4   const N_supports = 501
5   tf_lower = 1.0
6   tf_upper = 30.0
7   tf_start = 15.0
8   println("μ = ", μ, ", u_max = ", u_max, ", supports = ", N_supports)
9 end
10
11 # =====
12 # Cell 3: Compute L1 position
13 # =====

```

```
μ = 0.012150585609624, u_max = 0.1, supports = 501
```

```

1 begin
2   function d0omega_dx(x, μ)
3     r1 = abs(x + μ)
4     r2 = abs(x - (1 - μ))
5     return x - (1-μ)*(x + μ)/r1^3 - μ*(x - (1-μ))/r2^3
6   end
7
8   res = nlsolve(x -> d0omega_dx(x[1], μ), [0.84]; ftol=1e-14)
9   if !NLSolve.converged(res)
10     @warn "L1 solver did not converge!"
11   end
12   x_L1 = res.zero[1]
13   println("Computed L1 x = ", x_L1)
14 end
15
16 # =====
17 # Cell 4: Model + variables
18 # =====

```

```
Computed L1 x = 0.8369151257723574
```

An InfiniteOpt Model
 Feasibility problem with:
 Finite parameters: 0
 Infinite parameter: 1
 Variables: 10
 Derivatives: 0
 Measures: 0
 `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.LessThan{Float64}`: 3 constraints
 `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.EqualTo{Float64}`: 1 constraint
 `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.GreaterThan{Float64}`: 3 constraints
 Names registered in the model: tf, ux, uy, uz, vx, vy, vz, x, y, z, τ , $\partial\Omega_x$, $\partial\Omega_y$, $\partial\Omega_z$
 Transformation backend information:
 Backend type: TranscriptionBackend
 Solver: Ipopt
 Transformation built and up-to-date: false

```

1 begin
2     model = InfiniteModel(Ipopt.Optimizer)
3     @infinite_parameter(model,  $\tau \in [0, 1]$ , num_supports = N_supports)
4     @variable(model, tf_lower <= tf <= tf_upper, start = tf_start)
5
6     @variable(model, x, Infinite( $\tau$ ), start =  $\tau \rightarrow (-\mu + 0.01)*(1 - \tau) + x_{L1}*\tau$ )
7     @variable(model, y, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
8     @variable(model, z, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
9     @variable(model, vx, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
10    @variable(model, vy, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
11    @variable(model, vz, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
12
13    @variable(model, -u_max <= ux <= u_max, Infinite( $\tau$ ))
14    @variable(model, -u_max <= uy <= u_max, Infinite( $\tau$ ))
15    @variable(model, uz, Infinite( $\tau$ ))
16    fix.(uz, 0.0; force = true)
17
18    r1(x_, y_, z_) = sqrt((x_ +  $\mu$ )^2 + y_^2 + z_^2)
19    r2(x_, y_, z_) = sqrt((x_ - (1- $\mu$ ))^2 + y_^2 + z_^2)
20
21     $\partial\Omega_x(x_, y_, z_) = x_ - (1-\mu)*(x_ + \mu)/r1(x_, y_, z_)^3 - \mu*(x_ - (1-\mu))/r2(x_, y_, z_)^3$ 
22     $\partial\Omega_y(x_, y_, z_) = y_ * (1 - (1-\mu)/r1(x_, y_, z_)^3 - \mu/r2(x_, y_, z_)^3)$ 
23     $\partial\Omega_z(x_, y_, z_) = z_ * (1 - (1-\mu)/r1(x_, y_, z_)^3 - \mu/r2(x_, y_, z_)^3)$ 
24
25    model[: $\partial\Omega_x$ ] =  $\partial\Omega_x$ 
26    model[: $\partial\Omega_y$ ] =  $\partial\Omega_y$ 
27    model[: $\partial\Omega_z$ ] =  $\partial\Omega_z$ 

```

```

28     model
29 end
30
31 # =====
32 # Cell 5: Dynamics + boundary conditions + minimum-fuel objective
33 # =====

```

$$\int_{\tau \in [0,1]} \text{abs}(ux(\tau)) + \text{abs}(uy(\tau)) d\tau$$

```

1 begin
2     @constraint(model, ∂(x, τ) == tf * vx)
3     @constraint(model, ∂(y, τ) == tf * vy)
4     @constraint(model, ∂(z, τ) == tf * vz)
5     @constraint(model, ∂(vx, τ) == tf * (2*vy + model[:∂Qx](x,y,z) + ux))
6     @constraint(model, ∂(vy, τ) == tf * (-2*vx + model[:∂Qy](x,y,z) + uy))
7     @constraint(model, ∂(vz, τ) == tf * (model[:∂Qz](x,y,z) + uz))
8
9     @constraint(model, x(0) == -μ + 0.01)
10    @constraint(model, y(0) == 0)
11    @constraint(model, z(0) == 0)
12    @constraint(model, vx(0) == 0)
13    @constraint(model, vy(0) == 0)
14    @constraint(model, vz(0) == 0)
15
16    @constraint(model, x(1) == x_L1)
17    @constraint(model, y(1) == 0)
18    @constraint(model, z(1) == 0)
19    @constraint(model, vx(1) == 0)
20    @constraint(model, vy(1) == 0)
21    @constraint(model, vz(1) == 0)
22
23    @objective(model, Min, ∫(abs(ux) + abs(uy), τ))
24 end
25
26 # =====
27 # Cell 6: Solve
28 # =====

```

```
1 begin
2   set_silent(model)
3   @time optimize!(model)
4   println("Status: ", termination_status(model))
5   println("Solved tf = ", value(tf))
6 end
7
8 # =====
9 # Cell 7: Extract solution
10 # =====
```

```
*****
This program contains Ipopt, a library for large-scale nonlinear optimization.
Ipopt is released as open source code under the Eclipse Public License (EPL).
For more information visit https://github.com/coin-or/Ipopt
*****
```

```
50.740523 seconds (14.87 M allocations: 762.969 MiB, 0.19% gc time, 10.10% compilation time: 47% of which was recompilation)
Status: ITERATION_LIMIT
Solved tf = 26.238452348425152
```

```
1 begin
2   opt_x = value(x); opt_y = value(y); opt_z = value(z)
3   opt_vx = value(vx); opt_vy = value(vy); opt_vz = value(vz)
4   opt_ux = value(ux); opt_uy = value(uy); opt_uz = value(uz)
5   opt_tf = value(tf)
6   ts      = supports( $\tau$ ) .* opt_tf
7
8   println("Final pos error (x,y,z): ", opt_x[end]-x_L1, ", ", opt_y[end], ", ",
opt_z[end])
9   println("Final vel error (vx,vy,vz): ", opt_vx[end], ", ", opt_vy[end], ", ",
opt_vz[end])
10  println("Maximum |z| during transfer: ", maximum(abs, opt_z))
11 end
12
13 # =====
14 # Cell 8: Trajectory plots
15 # =====
```

```
Final pos error (x,y,z): -1.1102230246251565e-16, -9.197947074280056e-16, 0.0
Final vel error (vx,vy,vz): -2.0611068219074452e-16, 4.371737942774328e-16, 0.0
Maximum |z| during transfer: 0.0
```

```

1 begin
2   p_traj = plot(opt_x, opt_y, opt_z,
3               lw=2.5, label="Trajectory",
4               title="Earth → L1 with ONLY in-plane control (3D CR3BP)",
5               xlabel="x", ylabel="y", zlabel="z", camera=(30,30))
6   scatter!(p_traj, [-μ], [0], [0], marker=:circle, markersize=10, label="Earth")
7   scatter!(p_traj, [1-μ], [0], [0], marker=:star5, markersize=12, label="Moon")
8   scatter!(p_traj, [x_L1], [0], [0], marker=:diamond, markersize=10, color=:red,
9   label="L1")
10
11   p_xy = plot(opt_x, opt_y, lw=2,
12               title="x-y plane", aspect_ratio=:equal, legend=false)
13   scatter!(p_xy, [-μ], [0], marker=:circle, markersize=10)
14   scatter!(p_xy, [1-μ], [0], marker=:star5, markersize=12)
15   scatter!(p_xy, [x_L1], [0], marker=:diamond, markersize=10, color=:red)
16
17   p_z = plot(ts, opt_z,
18               title="Out-of-plane deviation z(t)", label="z", lw=2, color=:purple)
19   hline!(p_z, [0], linestyle=:dash, label="x-y plane")
20
21   p_ctrl = plot(ts, [opt_ux opt_uy opt_uz],
22               label=["ux" "uy" "uz (fixed=0)"],
23               title="Control inputs", ylim=(-0.11, 0.11))
24
25   display(p_traj)
26   display(p_xy)
27   display(p_z)
28   display(p_ctrl)
29 end
30 # =====
31 # Cell 9: Interval Tube Width Growth
32 # =====

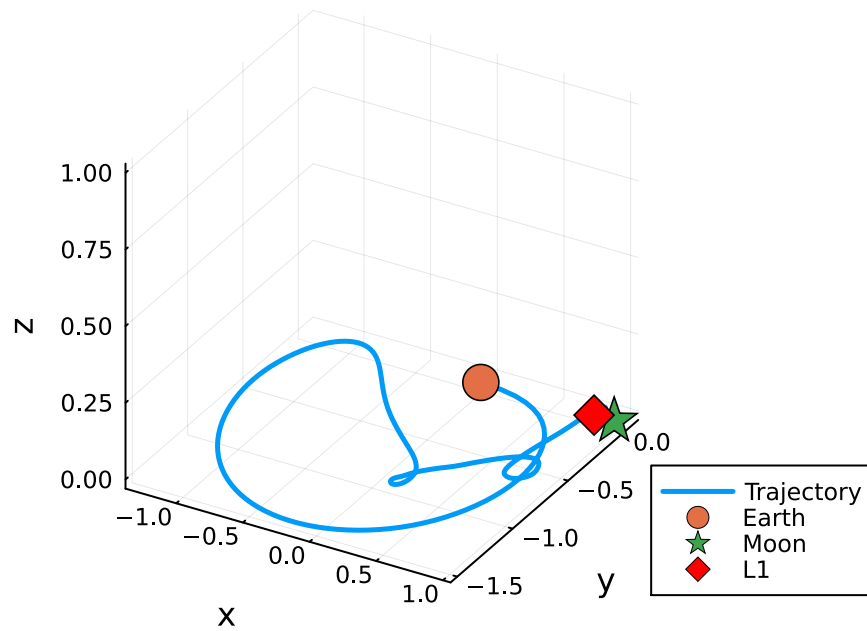
```

```

Plot{Plots.GRBackend() n=4}
Plot{Plots.GRBackend() n=4}
Plot{Plots.GRBackend() n=2}
Plot{Plots.GRBackend() n=3}

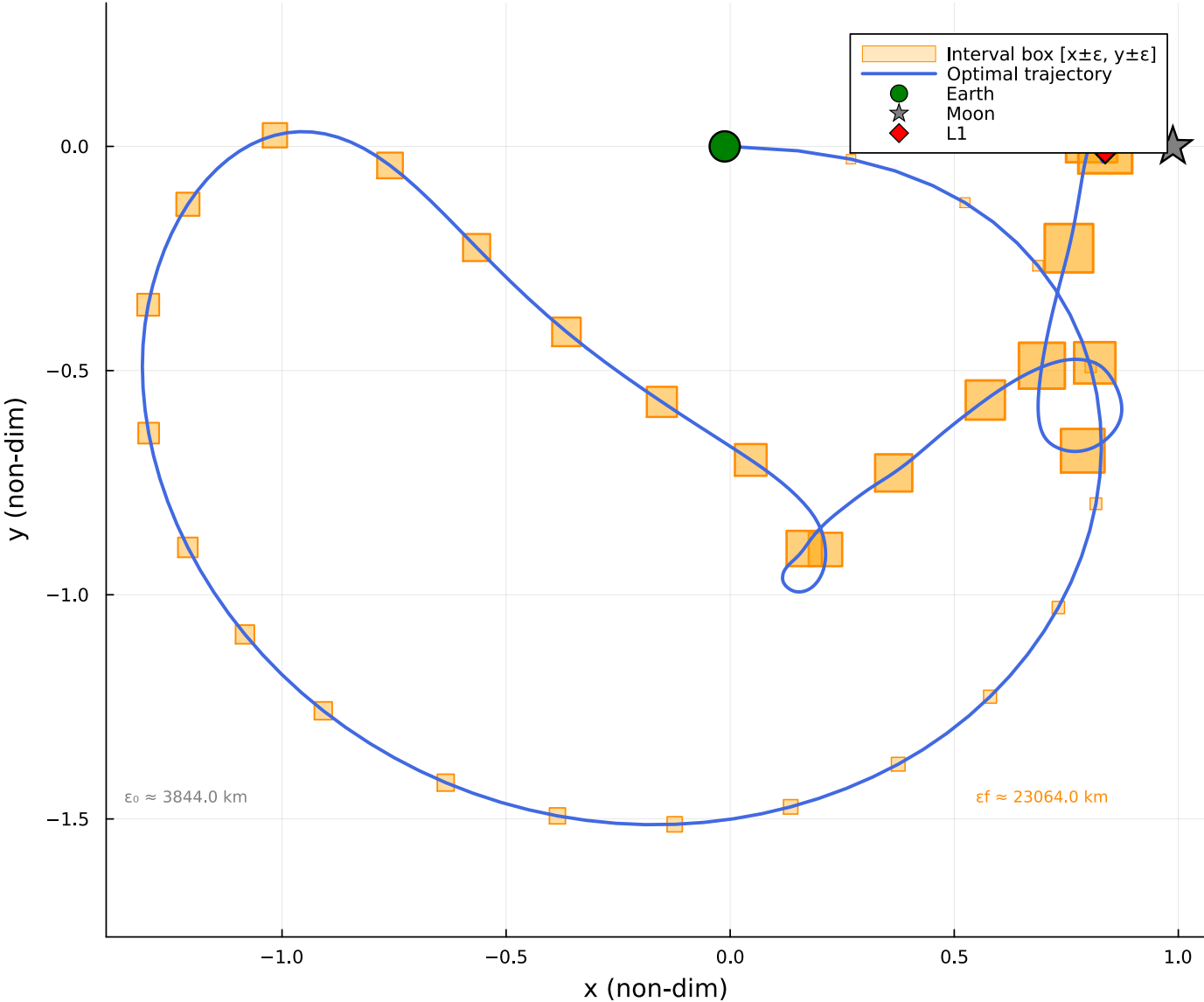
```


Earth → L1 with ONLY in-plane control (3D CR3BP)



```
1 p_traj
```

Earth → L1: Interval Reachability Tube (x-y plane)



```
1 ### Cell 9: Interval Reachability Boxes – Improved Visual
2
3 let
4   N = length(opt_x)
5
```

```

6      # — Cumulative arc length —————
7      arc = zeros(N)
8      for k in 2:N
9          arc[k] = arc[k-1] + sqrt((opt_x[k]-opt_x[k-1])^2 + (opt_y[k]-opt_y[k-1])^2)
10     end
11     total_arc = arc[end]
12
13     # — 35 evenly spaced boxes along arc length —————
14     n_boxes = 35
15     arc_tgts = range(0, total_arc, length=n_boxes)
16     box_idx = [argmin(abs.(arc .- s)) for s in arc_tgts]
17
18     # — Box size: exponential growth (matches instability near L1) —————
19     ε0 = 0.010
20     εf = 0.060
21     box_sizes = [ε0 * (εf/ε0)^((i-1)/(n_boxes-1)) for i in 1:n_boxes]
22
23     # — Plot —————
24     p = plot(title      = "Earth → L1: Interval Reachability Tube (x-y plane)",
25             xlabel      = "x (non-dim)",
26             ylabel      = "y (non-dim)",
27             legend      = :topright,
28             aspect_ratio = :equal,
29             size         = (800, 700))
30
31     # Draw boxes
32     for (i, k) in enumerate(box_idx)
33         cx = opt_x[k]
34         cy = opt_y[k]
35         b  = box_sizes[i]
36         t_n = (i - 1) / (n_boxes - 1)
37
38         rx = [cx-b, cx+b, cx+b, cx-b, cx-b]
39         ry = [cy-b, cy-b, cy+b, cy+b, cy-b]
40
41         plot!(p, rx, ry,
42              seriestype = :shape,
43              fillcolor  = :orange,
44              fillalpha  = 0.25 + 0.35 * t_n,
45              linecolor  = :darkorange,

```

```

46         linewidth = 0.6 + 1.2 * t_n,
47         label      = i == 1 ? "Interval box [ $x \pm \epsilon$ ,  $y \pm \epsilon$ ]" : ""
48     end
49
50     # Trajectory on top
51     plot!(p, opt_x, opt_y,
52           lw      = 2.2,
53           color    = :royalblue,
54           label    = "Optimal trajectory")
55
56     # Bodies – no zorder keyword (not supported in Plots.jl)
57     scatter!(p, [-μ], [0], marker=:circle, markersize=11,
58              color=:green, label="Earth")
59     scatter!(p, [1-μ], [0], marker=:star5, markersize=13,
60              color=:gray, label="Moon")
61     scatter!(p, [x_L1], [0], marker=:diamond, markersize=11,
62              color=:red, label="L1")
63
64     annotate!(p, -1.35, -1.45,
65              text("ε0 ≈ $(round(ε0*384400, digits=0)) km", 7, :left, :gray))
66     annotate!(p, 0.55, -1.45,
67              text("εf ≈ $(round(εf*384400, digits=0)) km", 7, :left, :darkorange))
68
69     println("=== Interval Reachability Boxes ===")
70     println("Boxes          : ", n_boxes)
71     println("ε0 (departure): ", round(ε0*384400, digits=0), " km")
72     println("εf (arrival)   : ", round(εf*384400, digits=0), " km")
73     println("Growth factor : ", round(εf/ε0, sigdigits=3), "x")
74
75     p
76 end

```

```

=== Interval Reachability Boxes ===
Boxes          : 35
ε0 (departure): 3844.0 km
εf (arrival)   : 23064.0 km
Growth factor  : 6.0x

```

